scientific reports

Check for updates

OPEN

Review of large YOLOv8 and RT-DETR energy efficiency on edge devices for real-time detection

Ivan Suchý & Michal Turčaník ✉

Deploying large object detectors on embedded edge platforms is governed by a joint trade-off among detection accuracy, end-to-end throughput, and total system power. This paper benchmarks the large variants YOLOv8l and RT-DETR-l across heterogeneous deployment runtimes on two edge platforms: Raspberry Pi 5 with CPU and NPU offload, and Nvidia Jetson Orin NX with GPU acceleration. Accuracy is evaluated on COCO val2017 using mAP50-95, while throughput and energy efficiency (FPS/W) are measured on a realistic end-to-end video pipeline that includes decoding, preprocessing, inference, and post-processing. Model-execution latency is analyzed separately from pipeline throughput to avoid ambiguity between inference time and end-to-end processing rate. On Raspberry Pi 5, CPU-only execution of large models is impractical due to multi-second per-frame latency, whereas NPU acceleration substantially improves energy efficiency for YOLOv8l, albeit with deployment constraints that can reduce accuracy. On Jetson Orin NX, TensorRT provides the strongest deployment path for both architectures; however, the relative ranking of YOLOv8l and RT-DETR-l depends on runtime realization rather than on nominal FLOPs alone. A generalized interpretation based on normalized latency and energy per nominal GFLOP, together with a decomposition of conversion sensitivity and quantization sensitivity, shows that deployment efficiency is jointly determined by nominal compute demand, memory-system behavior, runtime overhead, and accuracy retention after export and quantization. Under the tested conditions, none of the evaluated large-model configurations reaches the strict real-time target of 25 FPS for the full pipeline, indicating that further hardware-specific optimization and/or smaller model variants are still required for real-time edge deployment.

Keywords Computer graphics, Computer vision, Object detection, Tensors, Video signal processing, Neural nets, Data handling, Data visualisation, Embedded systems, Graphics processing units

Deploying deep neural networks on embedded platforms is fundamentally constrained by the need to balance detection accuracy with limited on-board compute and power budgets. This constraint is particularly relevant for vision-enabled UAVs and other autonomous edge systems, where object detection must operate under strict energy and thermal limits while preserving sufficient precision for reliable decision-making[1–5].

Within this context, both model architecture and deployment runtime are decisive. The YOLO family remains widely used because of its practical accuracy-speed balance across model sizes[6,7]. In parallel, transformer-based vision architectures have demonstrated strong representational capability through long-range feature interaction[8]. RT-DETR extends this direction to real-time object detection through a hybrid design that combines convolutional feature extraction with transformer-based interaction and decoder refinement[9]. However, deployment behavior on edge hardware cannot be inferred from parameter count or nominal FLOPs alone, since realized efficiency also depends on operator characteristics, runtime implementation, and hardware-specific optimization[10,11].

Although benchmarking studies on embedded object detection have expanded, important gaps remain. Existing works such as YOLOBench[6] and DeepEdgeBench[10] predominantly emphasize lightweight models for strongly resource-constrained devices. Comparisons between RT-DETR and YOLO-based detectors have also begun to appear[12,13], but they often focus on smaller variants or on desktop- and cloud-oriented execution. Consequently, there is still limited evidence on how large detectors with parameter counts above 30 million behave across heterogeneous edge deployments, especially when comparing CNN-dominated and transformer-hybrid architectures under CPU-, GPU-, and NPU-oriented runtimes.

Department of Informatics, Armed Forces Academy of general Milan Rastislav Štefánik, Liptovský Mikuláš, Slovakia. ✉email: michal.turcanik@aos.sk

To address this gap, this paper evaluates YOLOv8l and RT-DETR-l across multiple inference frameworks and deployment backends on two representative edge platforms: Raspberry Pi 5 with optional NPU acceleration and Nvidia Jetson Orin NX with GPU acceleration. Detection accuracy is assessed using mAP50-95 on COCO val2017, while deployment behavior is evaluated through end-to-end throughput, model-execution latency, and energy efficiency (FPS/W) on a realistic video-processing pipeline. In addition to direct benchmarking, the paper provides a deployment-oriented interpretation of architectural efficiency that goes beyond raw FPS and nominal FLOPs.

Related work

The rapid evolution of edge computing has shifted the focus of object detection research toward balancing high-precision architectures with the limited energy and computational budgets of mobile platforms, such as Unmanned Aerial Vehicles (UAVs) and autonomous systems.

Recent comparative studies emphasize that while newer models offer incremental gains, their deployment suitability depends heavily on hardware-specific optimization. Rey et al.[14] conducted a performance analysis of YOLOv8 variants on drones, identifying the Jetson Orin Nano as a superior platform over the Raspberry Pi 5 when utilizing INT8 quantization to maintain operational frame rates. This is complemented by the work of Alqahtani et al.[10], who benchmarked YOLOv8 against EfficientDet and SSD models across several generations of Raspberry Pi and Jetson boards. Their findings highlight that larger YOLO variants often struggle with real-time constraints on low-power devices unless coupled with specialized TPU accelerators.

The emergence of real-time transformers has introduced new competition for traditional CNNs. Miglionico et al.[13] performed the first direct comparison between YOLOv11 and RT-DETR for industrial safety tasks, evaluating their efficiency across PyTorch, OpenVINO, and TensorRT frameworks. Similarly, Mela and Sánchez[15] evaluated YOLO versions from v6 to v11 for maritime surveillance on Unmanned Surface Vehicles (USVs). Their results indicate that YOLOv8-based models often provide the most stable balance of architectural simplicity and throughput, maintaining a power profile below 2 W on the Orin Nano platform. Furthermore, Bemposta Rosende et al.[16] explored the use of YOLO and EfficientDet architectures for intelligent traffic management using UAVs, focusing on the trade-offs between training time and real-time inference on reduced-board computers.

In the field of underwater surveillance, Fan et al.[17] optimized YOLOv8n using neural architecture search (NAS) and channel-wise distillation (CWD), significantly outperforming baseline models in accuracy for marine organism detection. For urban applications, Sonnara et al.[18] introduced Light-Edge, a single-pass network for license plate recognition that uses a 1×1 channel-fusion block to halve power consumption compared to traditional anchor-based frameworks on the Jetson Nano.

A critical aspect of modern Edge AI is the impact of software overhead and extreme resource constraints. Wijaya et al.[19] investigated the energy consumption of YOLOv8 on the Nvidia Orin Nano, revealing that bare-metal execution offers a more energy-efficient profile than Docker containerization, a vital finding for long-endurance UAV missions. At the extreme end of the hardware spectrum, Faizi et al.[20] explored TinyML deployment on low-power microcontrollers, demonstrating that while pruning and knowledge distillation provide high compression, 8-bit quantization remains the most reliable strategy for maintaining detection precision.

Finally, the challenge of data availability has led to innovations in unsupervised learning. Ruiz-Barroso et al.[21] developed a real-time unsupervised video object detection system. By optimizing concurrent task execution across the CPU, GPU, and DLA units of the Jetson AGX Xavier, they achieved a 23.6× improvement in energy efficiency, signaling a shift toward hardware-aware architectures that can operate autonomously in dynamic environments without requiring pre-labeled datasets.

Inference frameworks and runtimes

Pytorch

PyTorch is a high-performance deep learning framework developed by Facebook AI Research. It is built around an imperative programming style, allowing computations to be executed dynamically and interactively. This makes it particularly well-suited for research and experimentation, where model structures may evolve iteratively. PyTorch provides automatic differentiation, GPU acceleration, and seamless integration with the Python ecosystem, enabling researchers to leverage a wide array of scientific computing tools. However, for deployment on edge devices, models must typically be converted to lighter runtimes such as TensorFlow Lite to reduce memory and computational overhead. While PyTorch now offers quantization tools and the ExecuTorch runtime for edge deployment, the standard PyTorch runtime remains resource-intensive compared to dedicated inference engines like TFLite or TensorRT.

TensorFlow lite

TensorFlow Lite (TFLite) is explicitly designed for deploying deep learning models on edge devices with limited computational resources. It utilizes various post-training quantization techniques, including dynamic range, integer, and float16 quantization, to reduce model size and computation requirements. These techniques enable efficient execution on CPUs, GPUs, and accelerators like Edge TPUs. Among them, integer quantization is particularly effective for edge devices, as it converts 32-bit floating-point parameters into an 8-bit representation, maintaining compatibility with integer-only hardware while significantly reducing memory and power consumption[22]. TFLite's architecture includes a converter that transforms trained TensorFlow models into a compressed FlatBuffer format and an interpreter that executes the quantized models. This setup supports low-latency inference, making TFLite suitable for real-time applications like image recognition and IoT systems.

Edge TPU

The Edge TPU, embedded in devices such as the Coral USB Accelerator, is designed to provide high-speed and energy-efficient inference for machine learning applications on edge devices. The architecture leverages a highly parallelized matrix multiplication engine, where computations are arranged in a grid-like structure to efficiently perform operations such as multiplying weights and inputs. This design minimizes latency and maximizes throughput, making the Edge TPU particularly suitable for tasks such as object detection and image classification in embedded systems. To ensure compatibility, models must be quantized and converted to Edge TPU-compatible TensorFlow Lite format. This involves a pre-compilation step using the Coral TPU compiler, which maps supported neural network operations to the TPU hardware. Studies have demonstrated the Edge TPU's capabilities across diverse convolutional neural networks, showcasing its high performance under constrained power and memory conditions. However, its architecture, designed to balance compute efficiency and energy use, requires precise model preparation to leverage its full potential[23,24].

HailoRT

HailoRT is the high-performance runtime software designed to deploy neural network models on Hailo-8 AI accelerators. It works in conjunction with the Hailo Dataflow Compiler, which translates models from standard frameworks (such as TensorFlow and ONNX) into a proprietary binary format optimized for the hardware's dataflow architecture. This software stack enables efficient resource management and low-latency inference, leveraging the NPU's 26 TOPS capability[25]. However, the compilation process faces significant challenges with transformer-based architectures like RT-DETR. The Dataflow Compiler is primarily optimized for static graph representations typical of CNNs and struggles to map the dynamic computational graphs and complex attention mechanisms of transformers onto the NPU's fixed-function elements. Consequently, while the software seamlessly integrates with platforms like the Raspberry Pi 5 via the AI HAT+, it currently limits successful deployment to CNN-based tasks, preventing the execution of unsupported transformer operations.

NCNN

NCNN, developed by Tencent, is a high-performance neural network inference framework optimized for mobile platforms, including mobile phones, embedded devices, and IoT systems. Implemented entirely in C + + and without dependencies on third-party libraries, NCNN ensures compatibility across various operating systems, including Linux, Android, iOS, and macOS. Its portability and flexibility make it a robust choice for deploying deep learning models in resource-constrained environments. Key features of NCNN include support for quantization, which reduces the precision of model weights and activations to lower memory usage and improve runtime performance without significant loss of accuracy. Additionally, NCNN is compatible with major deep learning frameworks such as TensorFlow, Caffe, and ONNX, simplifying the process of deploying pre-trained models[26]. NCNN is recognized for its rapid processing capabilities on mobile CPUs, leveraging multi-core parallel computing and efficient resource management, such as large and small core scheduling on ARM-based architectures. It also supports GPU acceleration using the Vulkan interface, allowing faster image processing and real-time performance on embedded devices[27].

TensorRT

TensorRT, developed by NVIDIA, is a high-performance inference optimization engine designed for GPU-based devices, including edge platforms like the Jetson series. It optimizes deep learning models using techniques such as 8-bit precision calibration, while also supporting float32 and float16 formats for tasks requiring higher computational precision or compatibility with specific operations. TensorRT employs layer and tensor fusion, kernel auto-tuning, and dynamic tensor memory allocation to minimize latency. Furthermore, it supports multi-stream execution, enabling efficient processing of parallel inputs[28]. These optimizations ensure maximum GPU utilization and reduced memory overhead, making TensorRT particularly suitable for resource-constrained edge devices. Additionally, TensorRT engines compress and map models to the hardware's CUDA capabilities, leveraging multi-threading and stream concurrency to enhance performance and efficiency. TensorRT is seamlessly integrated into NVIDIA's software stack, including CUDA and cuDNN, which provides low-level GPU kernel optimizations for accelerated matrix operations. The SDK also includes tools for ONNX model conversion and supports a wide range of frameworks, enabling developers to optimize and deploy pre-trained models with minimal overhead. Its support for dynamic batching further enhances its ability to handle high-throughput inferencing in production environments while maintaining real-time performance.

MNN

MNN (Mobile Neural Network) is a lightweight deep learning inference engine developed by Alibaba, specifically optimized for mobile and embedded devices. Unlike general-purpose frameworks, MNN focuses on minimizing memory footprint and maximizing computational efficiency through operator fusion and memory reuse strategies. It supports a wide range of hardware backends, including ARM CPUs, GPUs (via OpenCL/Vulkan), and NPUs, making it highly versatile for heterogeneous edge platforms. Studies have shown that MNN often outperforms TFLite in CPU-based inference latency on ARM architectures due to its highly optimized assembly kernels for convolution operations. Its architecture allows for the conversion of models from TensorFlow, PyTorch, and Caffe, facilitating the deployment of complex models like YOLO on resource-constrained hardware without significant accuracy loss.

PaddlePaddle

PaddlePaddle (Parallel Distributed Deep Learning) is an industrial-grade deep learning platform developed by Baidu, which includes the Paddle Lite inference engine for edge deployment. It is characterized by its

support for both dynamic and static graphs, allowing for flexible model development and efficient deployment. Paddle Lite is optimized for ARM-based processors and integrates closely with hardware accelerators, utilizing techniques such as kernel fusion and precision calibration (INT8) to enhance throughput. Research indicates that PaddlePaddle's inference engine is particularly effective in maintaining high accuracy-speed trade-offs for object detection tasks on embedded systems, often rivaling TensorRT in specific GPU-accelerated scenarios on Jetson platforms. Its model compression toolkit (PaddleSlim) further aids in reducing model size for efficient storage and transmission in IoT networks.

Methodology

Pre-trained models YOLOv8 and RT-DETR

The YOLO series has undergone continuous development, with each iteration refining its architecture to improve accuracy and efficiency. YOLOv8, the version evaluated in this study, introduces significant enhancements such as an anchor-free detection head and optimized backbone and neck designs. These architectural improvements enhance feature extraction and object detection performance, contributing to higher precision and speed[29]. However, these advancements come with varying computational demands, particularly in feature processing stages. The anchor-free architecture simplifies the detection pipeline by eliminating predefined anchor boxes, yet it necessitates more precise feature mapping and intensive computation during training and inference. YOLOv8 is available in multiple model sizes *nano* (*n*), *small* (*s*), *medium* (*m*), *large* (*l*), and *extra-large* (*x*) where size refers to the number of parameters and layers within the architecture. Smaller models prioritize speed and energy efficiency, making them more suitable for deployment on constrained hardware, while larger models offer superior accuracy at the cost of higher computational and energy demands. The most energy-intensive operations in YOLOv8 occur within convolutional layers and upsampling components in the neck architecture, where substantial memory access and matrix computations are required.

RT-DETR represents a paradigm shift from conventional CNN-based object detection to transformer-based architectures, offering a novel approach to processing multiscale features. The model's hybrid encoder integrates Intra-scale Feature Interaction (AIFI) with the CNN-based Cross-scale Feature Fusion (CCFF), resulting in a significant improvement in detection accuracy[9]. However, the reliance on transformer mechanisms introduces substantial computational overhead[11]. The attention mechanisms, which rely on large-scale matrix multiplications and require extensive memory bandwidth for feature interactions, make RT-DETR inherently more energy-demanding compared to traditional CNN-based detectors. RT-DETR is available in two configurations *large* (*l*) and *extra-large* (*x*) which differ in the number of parameters and architectural complexity. While these larger configurations deliver higher accuracy, they also impose considerable computational and energy requirements, particularly during attention-based feature encoding and query optimization in the decoder. The iterative decoder, designed to refine object queries, further contributes to the computational intensity, especially when processing high-resolution inputs.

In this study, the large variants YOLOv8l and RT-DETR-l were selected for evaluation and initialized with weights pre-trained on the COCO dataset. These variants were chosen due to their comparable parameter counts to enable a balanced assessment, and to examine whether parameter-intensive detectors can be deployed effectively under edge-device constraints. Accuracy (mAP) was evaluated on the standard COCO val2017 validation set (5,000 images across 80 categories). Throughput and energy efficiency were measured on the MOT16-13 benchmark video (750 frames, 1920×1080 at 25 FPS), which represents a high-bitrate workload and includes realistic video decoding and data-movement overheads in the Total System Power measurements.

The evaluation protocol focused on the direct inference performance of the pre-trained models. Instead of additional fine-tuning, the models were exported directly from the Ultralytics framework into hardware-optimized formats. Accuracy was verified post-export to quantify the impact of framework-specific optimizations and quantization on the original COCO-trained weights. This methodology ensures that the reported results reflect the models' inherent capabilities as provided by the standard architectures.

Table 1 summarizes the key specifications of the pre-trained YOLOv8l and RT-DETR-l models used in this study. These metrics capture critical attributes, including the input resolution, parameter count, and FLOPs per image, providing context for the expected inference cost across different runtime environments. YOLOv8 and RT-DETR Performance Metrics.

Models accuracy

Model accuracy was evaluated using the standard Mean Average Precision (mAP) metric on the COCO val2017 dataset. Specifically, we report mAP@50–95 (the primary challenge metric), mAP@50, and mAP@75 to provide a comprehensive view of detection performance across different IoU thresholds.

Tables 2 and 3 present the precision metrics (mAP 50–95, mAP 50, and mAP 75) for each model configuration after exporting to the respective inference formats. To ensure high-throughput processing of the validation dataset, all precision metrics were verified within the Google Colab environment using high-performance cloud-based GPU instances. To ensure technical consistency, precision validation for CPU-based and NPU-

Model	Size (pixels)	Params (M)	FLOPs (B)
YOLOv8l	640	43.63	165.4
RT-DETR-l	640	32.8	108

Table 1. YOLOv8 and RT-DETR performance metrics YOLOv8 and RT-DETR performance metrics.

Framework	mAP50-95	mAP50	mAP75
PyTorch	0.524	0.706	0.568
TFLite (float32)	0.524	0.706	0.567
TensorRT (float32)	0.507	0.699	0.549
TensorRT (int8)	0.492	0.689	0.531
MNN	0.524	0.706	0.568
MNN (int8)	0.523	0.705	0.566

Table 2. RT-DETR model precision (Validated in Google Colab Environment).

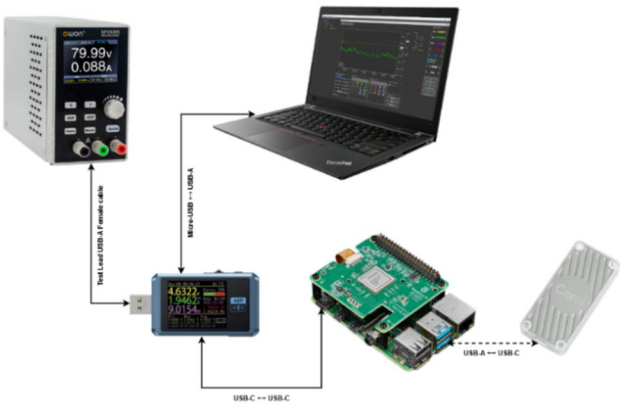


Fig. 1. Connection diagram for measurements of Raspberry Pi 5.

accelerated formats (Edge TPU, Hailo-8) was conducted on the Raspberry Pi 5, while GPU-accelerated formats (TensorRT) were validated on the Nvidia Jetson Orin NX platform. These metrics provide a standardized baseline for comparing how different runtime engines and quantization levels affect detection precision relative to the original pre-trained weights.

Experimental pipeline

Experiments were conducted on two edge platforms: the Raspberry Pi 5 (8GB) running Debian GNU/Linux 12 (kernel 6.6.62) and the Nvidia Jetson Orin NX (16GB) running Linux for Tegra (L4T) 36.4.0 with JetPack 6.1. The Raspberry Pi 5 was tested with two accelerators: the Coral USB Accelerator and the Raspberry Pi AI HAT+. The Jetson Orin NX was configured in MAXN power mode to utilize full computational capabilities. Software environments included Ultralytics 8.3, TensorFlow 2.17/2.16, PyTorch 2.5, and specialized runtimes (TensorRT, EdgeTPU, HailoRT, NCNN). While device-specific optimizations like overclocking or *jetson_clocks* were available, standard factory configurations were used to ensure reproducibility.

Throughout this paper, real-time denotes sustained end-to-end video processing throughput of ≥ 25 FPS for the evaluated pipeline (video decoding + pre-processing + inference + post-processing). This corresponds to an average per-frame processing budget of ≤ 40 ms. Configurations below 25 FPS are therefore treated as non-real-time for this workload. Inference was executed using the standard Ultralytics Python API, ensuring streamlined integration with hardware backends and consistent preprocessing across all tested configurations. During the execution of this script, custom Python scripts were concurrently run to monitor computational resource utilization. For the Jetson Orin NX, these scripts tracked both CPU and GPU usage, utilizing the *jtop* tool from the *jetson-stats* Python package. Resource monitoring on the Raspberry Pi 5 was managed by executing the command-line tool *mpstat* in parallel, which logged CPU utilization values into a separate output file.

The experimental setup utilizing a Raspberry Pi 5 was further enhanced with two hardware accelerators: a Coral USB Accelerator and a Raspberry Pi AI HAT+, both functioning as coprocessors to enable high-speed inference. These accelerators were used only with the YOLOv8 model, which was exported to TensorFlow Lite, INT8-quantized, and compiled for Edge TPU deployment. RT-DETR could not be deployed on Edge TPU or Hailo-8 due to conversion and operator-support limitations for transformer-based architectures (see Section III-D).

In addition to video-based object detection inference, power consumption was measured for both models across all tested runtime environments on both devices. An OWON SPE8205 programmable DC power supply was employed to provide stable voltage regulation to the Raspberry Pi 5, while a Fnirsi FNB48P USB power meter was integrated in-line to capture continuous power consumption data during inference operations. The complete experimental setup for power measurement on the Raspberry Pi 5 platform is illustrated in Fig. 1.

The Fnirsi FNB48P USB power meter employed in this study utilizes a high-resolution 16-bit ADC for precise voltage and current measurements, with configurable data logging intervals via the accompanying PC software interface. According to the device specifications, the meter provides voltage measurement accuracy of

±(0.2‰ + 2) with a resolution of 0.00001 V in the 4–24 V range, and current measurement accuracy of ±(0.5‰ + 2) with a resolution of 0.00001 A in the 0–6.5 A range. Power measurements are derived from the instantaneous voltage and current readings, supporting low-speed waveform sampling rates from 2 sps to 100 sps for detailed temporal analysis.

For the Raspberry Pi 5, power consumption measurements were conducted sequentially for each accelerator configuration. Specifically, measurements were performed separately for the Raspberry Pi 5 with the Google Coral USB Accelerator and the Raspberry Pi AI HAT+, ensuring independent characterization of each accelerator's power profile. This methodological approach provides a comprehensive assessment of the Total System Power (TSP). All reported power values represent the complete electrical load of the platform (including CPU, memory, storage, and operating system overhead) plus the consumption of any external AI accelerators. This approach was chosen to ensure physical interpretability in the context of UAV deployments, where the critical constraint is the total drain on the onboard battery rather than isolated component consumption. The power meter recorded cumulative energy consumption throughout each inference session, enabling calculation of total energy expenditure per test configuration. Data was continuously logged via USB connection to a host computer during the entire inference process, with measurements synchronized to the video processing timeline to ensure accurate correlation between computational workload and power consumption.

The energy consumption of the models was measured in discrete time steps using Total System Power readings P(t_i). The total energy E_{total} consumed during an inference run was calculated as:

$$E_{total} = \sum_{i=1}^{n} P(t_i)\,\Delta t \tag{1}$$

where $P(t_i)$ is the instantaneous power at time t_i, and Δt is the sampling interval. For each run, the average power was computed as $P_{avg} = E_{total}/T$, where T is the run duration. The throughput was measured as FPS.

Energy efficiency is reported in frames per second per watt (FPS/W) and is defined as:

$$Efficiency = \frac{FPS}{P_{avg}} \tag{2}$$

To ensure terminological consistency in the subsequent analysis, two performance levels are distinguished throughout the remainder of the paper. The metrics FPS and FPS/W refer to end-to-end pipeline performance, measured on the MOT16-13 video stream and therefore including decoding, preprocessing, inference, post-processing, and the corresponding Total System Power of the complete platform. By contrast, the reported inference time (ms) refers to model-execution latency as exposed by the evaluated runtime environment. This distinction is essential because a configuration may exhibit improved model-level latency without achieving proportionally higher end-to-end throughput once video I/O, host-side processing, and deployment-runtime overhead are included. Accordingly, Section V reports both views of performance, while Section VI uses this distinction to interpret deployment efficiency beyond nominal FLOPs.

Results

The results are presented from three complementary perspectives: model-execution latency, end-to-end deployment efficiency, and the resulting accuracy-efficiency trade-offs. This structure reflects the fact that the evaluated configurations differ not only in nominal model workload, but also in runtime realization, accelerator support, and full-pipeline overhead under realistic edge deployment conditions.

Inference time

The experimental data recorded during the testing phase reveal substantial performance gaps between standard frameworks and hardware-optimized runtimes. As illustrated in Fig. 2, the Raspberry Pi 5 platform faces significant bottlenecks when executing large model variants using PyTorch or TensorFlow Lite float32, where inference times exceed several seconds per frame. In contrast, specialized hardware acceleration via the Hailo-8 dramatically reduces latency and achieves the most efficient execution path for YOLOv8l on this platform.

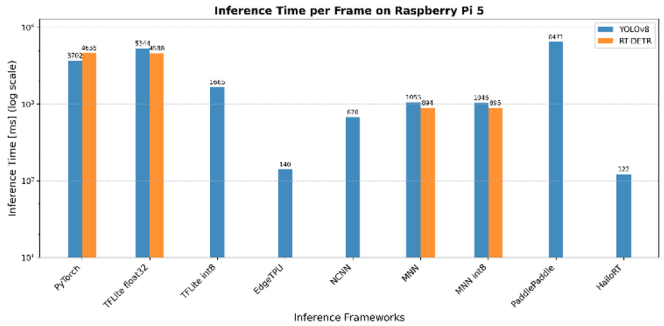


Fig. 2. Inference time on Raspberry Pi 5.

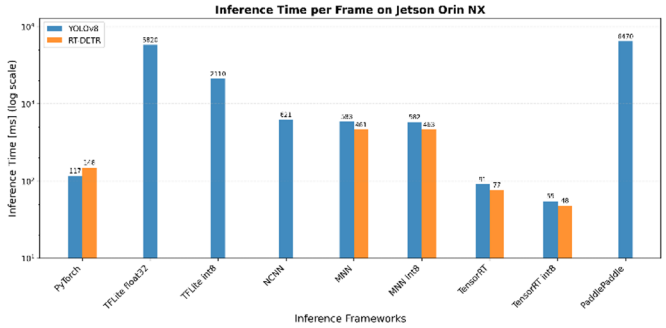


Fig. 3. Inference time on Jetson Orin NX.

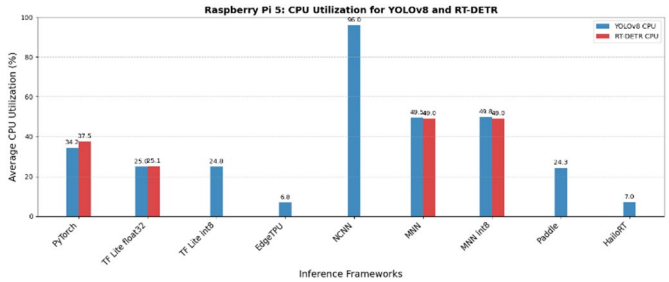


Fig. 4. CPU utilization during inference on Raspberry Pi 5.

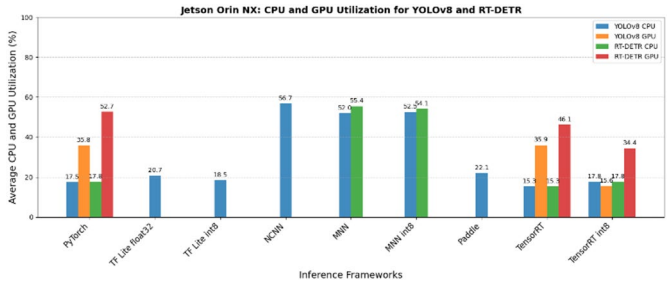


Fig. 5. CPU and GPU utilization during inference on Jetson Orin NX.

On the Jetson Orin NX platform, the impact of architectural optimization and reduced numerical precision is evident. As shown in Fig. 3, TensorRT substantially improves model-execution latency relative to research-oriented runtimes. While TensorFlow Lite implementations on Jetson remain prohibitively slow in CPU-bound execution, TensorRT uses the available CUDA and Tensor cores effectively and maintains sub-100 ms inference times for the evaluated large-model configurations. These results indicate that, for Jetson-class GPU devices, deployment efficiency depends strongly on the runtime's ability to compile and map the model graph to hardware-specific execution kernels.

Resource utilization also differs markedly across the tested environments. On Raspberry Pi 5, Fig. 4 indicates that NCNN induces the highest CPU stress, nearly saturating the multicore capabilities of the Broadcom processor. In contrast, specialized runtimes for the Edge TPU and Hailo-8 offload a substantial portion of the workload and therefore reduce host-side CPU utilization during active inference.

For the GPU-accelerated Jetson Orin NX, Fig. 5 highlights the efficiency of the TensorRT engine. While PyTorch shows only moderate GPU involvement, TensorRT INT8 achieves a dominant GPU utilization profile, reaching approximately 90% for both YOLOv8l and RT-DETR-l. This balanced resource distribution is important for edge devices because it reduces CPU-side pressure while maximizing the throughput of the parallel processing units. By contrast, NCNN on Jetson does not exploit GPU acceleration effectively and remains limited by CPU-bound execution.

Energy efficiency

The energy-efficiency analysis in FPS/W shows that model architecture and runtime choice must be interpreted together with total system power. On Raspberry Pi 5, shown in Figs. 6 and 7, the Hailo-8 accelerator delivers

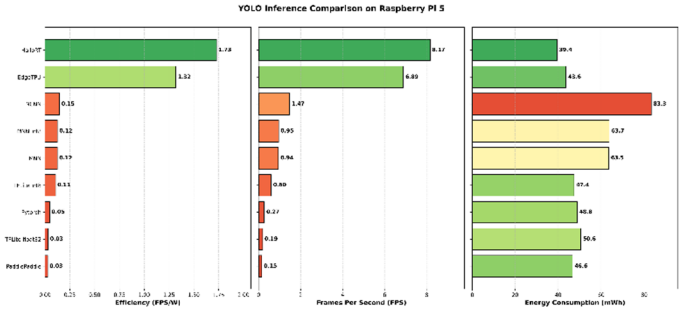


Fig. 6. YOLO Efficiency (FPS/W) on Raspberry Pi 5.

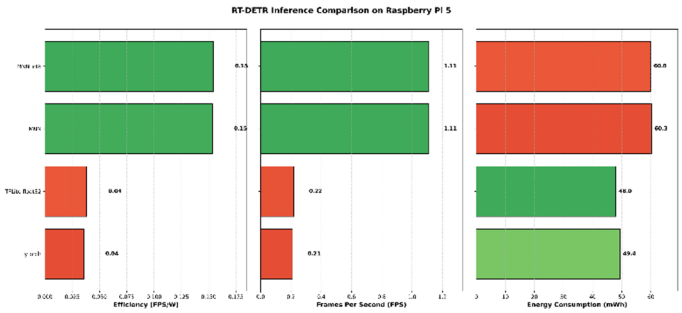


Fig. 7. RT-DETR Efficiency (FPS/W) on Raspberry Pi 5.

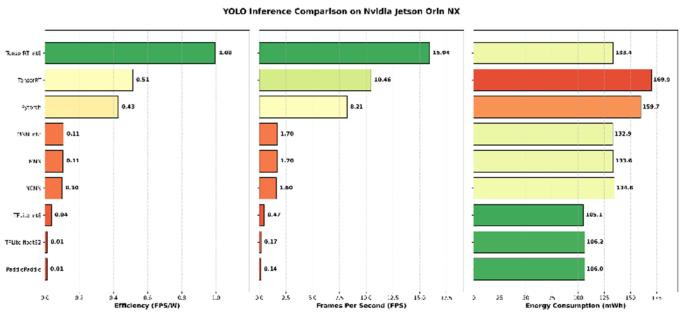


Fig. 8. YOLO Efficiency (FPS/W) on Nvidia Jetson Orin NX.

the highest measured efficiency for YOLOv8l, reaching 1.7281 FPS/W and establishing the strongest practical operating point on that platform under the tested conditions. In contrast, CPU-based execution remains highly inefficient, with multi-second latency and efficiency dropping as low as 0.0313 FPS/W. These results confirm that large detectors are not practically deployable on Raspberry Pi 5 in CPU-only form when sustained video throughput and energy constraints are both relevant.

On the Jetson Orin NX platform, shown in Figs. 8 and 9, the advantage of GPU-optimized runtimes is even more explicit. The highest efficiency values are consistently achieved through TensorRT with INT8 quantization, reaching 0.9960 FPS/W for YOLOv8l and 1.0597 FPS/W for RT-DETR-l. These results show that for large models, deployment success depends not only on nominal computational demand but also on whether the runtime can eliminate host-side overhead and exploit the target accelerator efficiently. They also highlight that hardware-specific constraints, such as the limited on-chip SRAM of the Edge TPU, may be just as decisive for practical deployment as raw arithmetic throughput.

Accuracy-efficiency trade-off analysis

To compare deployment options on a common decision criterion, configurations with mAP50–95 below 0.50 are treated as accuracy-limited for practical use. Within this constraint, Fig. 10 highlights the Pareto frontier of non-dominated accuracy-efficiency trade-offs under the tested conditions. Since none of the evaluated large-model deployments reached the strict real-time requirement of at least 25 FPS for the full pipeline, the Pareto frontier represents the best attainable accuracy-efficiency combinations within a sub-real-time operating regime.

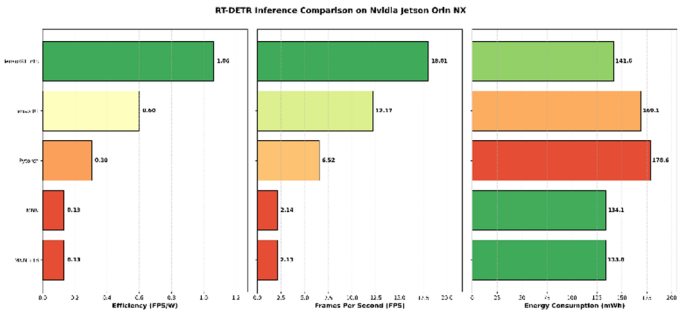


Fig. 9. RT-DETR Efficiency (FPS/W) on Nvidia Jetson Orin NX.

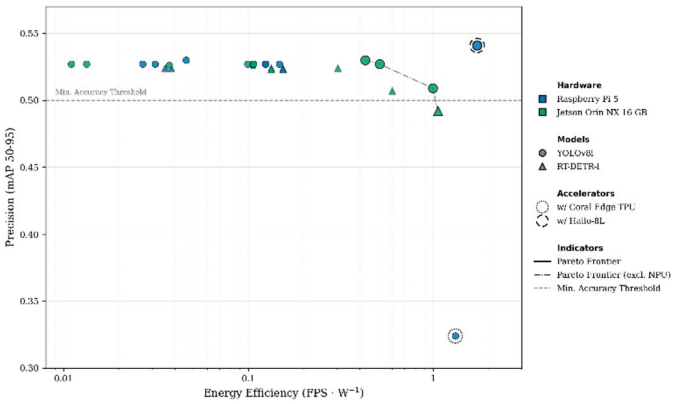


Fig. 10. Pareto frontier of detection accuracy versus energy efficiency across embedded platforms.

A particularly important finding is the efficiency trap associated with specialized NPU accelerators on Raspberry Pi 5. Both Hailo-8 and Edge TPU exceed 1.3 FPS/W, but their impact on accuracy differs substantially. HailoRT preserves high precision at 0.541 mAP50–95, whereas the Edge TPU configuration falls well below the practical threshold, dropping to 0.324 mAP50–95. This reduction is linked to mandatory INT8 quantization and the required decrease in input resolution to 224×224 in order to satisfy the Edge TPU memory constraint. The result shows that very high FPS/W alone is not a sufficient indicator of deployment quality when accuracy degradation makes the detector unsuitable for demanding real-world tasks.

Tables 4, 5, 6 and 7 provide the detailed numerical values corresponding to the measurements shown in the figures, including frames per second, average power consumption, inference time, and FPS/W for all evaluated configurations.

Generalized interpretation of architectural efficiency

Section V showed that the relative behavior of YOLOv8l and RT-DETR-l changes across deployment runtimes. The following analysis therefore separates nominal workload, runtime realization, and post-export accuracy retention in order to explain the measured differences more systematically.

The framework introduced below is intended as a generalized explanatory interpretation for edge-based object detectors. It should be understood as an analytical model for interpreting the measured results, not as a direct profiler-based attribution of every kernel-level contribution.

Decomposition of edge inference efficiency beyond FLOPs

To avoid ambiguity, two distinct time quantities are used. Let T_{e2e} denote the end-to-end time per processed frame for the full pipeline used in this study, i.e., video decoding, preprocessing, inference, and post-processing:

$$T_{e2e} = \frac{1}{FPS} \tag{3}$$

Let T_{infer} denote the model inference time reported in Tables 6 and 7. At the model-execution level, the measured inference time can be approximated as

$$T_{infer} \approx T_{comp} + T_{mem} + T_{rt} \tag{4}$$

where T_{comp} is the compute-dominated component, T_{mem} captures memory-system effects such as intermediate tensor movement and synchronization, and T_{rt} denotes runtime overhead associated with graph execution, kernel launch granularity, operator scheduling, and memory planning.

The end-to-end energy per processed frame is

$$E_{\text{frame}} = P_{avg} T_{e2e} = \frac{P_{\text{avg}}}{FPS} \tag{5}$$

where P_{avg} is the measured average total system power.

Let C denote the nominal GFLOPs per image from Table 1. To compare model families more informatively than by raw FPS alone, two normalized indicators are introduced:

$$cL_C = \frac{T_{infer}}{C} \quad [ms/GFLOP] \tag{6}$$

$$E_C = \frac{E_{frame}}{C} = \frac{P_{avg}}{FPS\,.C} \quad [mJ/GFLOP] \tag{7}$$

The quantity L_C expresses model-only inference latency per nominal unit of work, whereas E_C represents end-to-end energy per nominal unit of work. Taken together, they distinguish between computational efficiency at the model-execution level and energy efficiency at the full-pipeline level.

Table 8 shows that RT-DETR-l exhibits higher L_C and higher E_C than YOLOv8l across the evaluated Jetson runtimes. Thus, even when RT-DETR-l becomes faster in raw FPS under TensorRT because of its lower total nominal FLOP count, it remains costlier per nominal unit of work according to both normalized indicators.

Architecture-level sources of latency and energy

The systematic difference in L_C and E_C between the two detectors is consistent with their operator composition. YOLOv8l is dominated by convolutional processing, which on embedded GPUs generally benefits from local spatial reuse, regular memory-access patterns, and extensive kernel or layer fusion. RT-DETR-l, in contrast, combines convolutional feature extraction with transformer-based interaction through the hybrid encoder based on AIFI and CCFF[9]. This design can reduce network-level nominal FLOPs relative to a purely convolutional large detector, but it also introduces global token interaction whose computational structure differs from standard convolutions[8,11].

For the 640 × 640 input used in this study, the stride-32 feature map has spatial size 20 × 20, corresponding to $N = 400$tokens. In standard dense self-attention, the attention-score tensor scales as $O\left(N^2\right)$[30]. Even at this coarse scale, one score matrix contains $N^2 = 400^2 = 160{,}000$ elements per head. In implementations that explicitly materialize attention scores, several heads can therefore produce temporary score tensors in the MB range, depending on precision and kernel design. By contrast, convolutions preserve local connectivity and avoid quadratic pairwise expansion over all tokens.

The implication is not that the attention block necessarily dominates the total FLOP count of the complete detector. Rather, transformer-style global interaction introduces intermediate representations whose realized cost can depend more strongly on memory movement and runtime implementation than purely local convolutions. In this sense, nominal FLOPs alone do not fully capture the realized latency and energy cost of the architecture on an embedded GPU.

Runtime realization and why the ranking changes across frameworks

Because the Jetson ranking flips between PyTorch and TensorRT, the metrics below quantify how strongly each model benefits from compiled execution and INT8 optimization.

To quantify the benefit of runtime optimization at the model-execution level, the normalized compile gain and normalized INT8 gain are defined as

$$R_{compile} = \frac{L_{C,\,Pytorch}}{L_{C,TRT,FP32}} \tag{8}$$

$$R_{INT8} = \frac{L_{C,\,TRT,FP32}}{L_{C,TRT,INT8}} \tag{9}$$

A larger $R_{compile}$ indicates greater sensitivity to graph compilation and low–level runtime optimization. In the present measurements, RT–DETR – l exhibits the larger $R_{compile}$, which indicates stronger benefit from compiled execution. This interpretation is consistent with TensorRT mechanisms such as graph optimization, operator fusion, kernel auto-tuning, and memory planning[28]. However, even after these optimizations, RT-DETR-l remains less efficient per nominal GFLOP than YOLOv8l according to Table 8. Once runtime overhead is sufficiently reduced, its lower total nominal FLOP count becomes more influential in raw FPS, which explains why RT-DETR-l overtakes YOLOv8l under TensorRT.

Separating conversion sensitivity from quantization sensitivity

The accuracy measurements in Tables 2 and 3 permit a more rigorous interpretation than a single PyTorch-to-INT8 comparison. In particular, deployment-related accuracy loss can be decomposed into a conversion component and a quantization component:

Framework	mAP50-95	mAP50	mAP75
PyTorch	0.530	0.698	0.577
TFLite (float32)	0.527	0.696	0.574
TFLite (int8)	0.526	0.695	0.573
TensorRT (float32)	0.527	0.696	0.574
TensorRT (int8)	0.509	0.678	0.558
NCNN	0.527	0.696	0.574
MNN	0.527	0.696	0.574
MNN (int8)	0.527	0.696	0.574
PaddlePaddle	0.527	0.696	0.574
EdgeTPU	0.324	0.468	0.354
HailoRT	0.541	0.708	0.592

Table 3. YOLOv8 model precision (Validated in Google Colab Environment) *Note* EdgeTPU results were obtained using a reduced input resolution of 224 × 224; all other reported results use 640 × 640 unless stated otherwise.

Inference Format	FPS	Average Power (W)	Inference time (ms)	Efficiency (FPS/W)
Pytorch	0.27	5.86	3702	0.0461
TFLite (float32)	0.19	6.07	5344	0.0313
TFLite (int8)	0.60	5.68	1665	0.1056
NCNN	1.47	9.99	676	0.1471
MNN	0.94	7.62	1055	0.1234
MNN (int8)	0.95	7.65	1046	0.1242
PaddlePaddle	0.15	5.59	6471	0.0268
EdgeTPU	6.89	5.23	140	1.3172
HailoRT	8.17	4.73	122	1.7281

Table 4. YOLOv8 inference efficiency on Raspberry Pi 5. *Note* EdgeTPU results were obtained using a reduced input resolution of 224 × 224; all other reported results use 640 × 640 unless stated otherwise.

Inference Format	FPS	Average Power (W)	Inference time (ms)	Efficiency (FPS/W)
Pytorch	0.21	5.93	4655	0.0354
TFLite (float32)	0.22	5.76	4588	0.0382
MNN	1.11	7.23	894	0.1534
MNN (int8)	1.11	7.20	895	0.1542

Table 5. RT-DETR inference efficiency on Raspberry Pi 5.

$$cS_{conv} = \frac{A_{Pytorch} - A_{TRT,FP32}}{A_{Pytorch}} \tag{10}$$

$$S_q = \frac{A_{TRT,FP32} - A_{TRT,INT8}}{A_{TRT,FP32}} \tag{11}$$

where A denotes mAP50–95.

This decomposition separates the loss introduced during export and compiled-runtime conversion from the additional loss introduced by INT8 quantization. The results are summarized in Table 9.

Table 9 shows that, in the tested TensorRT workflow, RT-DETR-l experiences a larger relative loss in the PyTorch-to-TensorRT FP32 step, whereas YOLOv8l loses a larger fraction in the subsequent FP32-to-INT8 step. Therefore, the larger overall PyTorch-to-TensorRT INT8 accuracy reduction observed for RT-DETR-l should not be attributed to quantization alone; it reflects the combined effect of conversion and quantization.

Generalized implications for edge detector selection

The analysis indicates that deployment efficiency is governed jointly by nominal workload, memory-related execution effects, runtime overhead, and accuracy retention after conversion and quantization. In practical edge deployment, none of these dimensions should be interpreted in isolation.

Inference Format	FPS	Average Power (W)	Inference time (ms)	Efficiency (FPS/W)
Pytorch	8.21	19.17	117	0.4283
TFLite (float32)	0.17	12.74	5820	0.0133
TFLite (int8)	0.47	12.62	2110	0.0373
TensorRT (float32)	10.46	20.39	91	0.5130
TensorRT (int8)	15.94	16.00	55	0.9960
NCNN	1.60	16.15	621	0.0991
MNN	1.70	16.03	583	0.1060
MNN (int8)	1.70	15.95	582	0.1066
PaddlePaddle	0.14	12.72	6470	0.0110

Table 6. YOLOv8 inference efficiency on Jetson Orin NX.

Inference Format	FPS	Average Power (W)	Inference time (ms)	Efficiency (FPS/W)
Pytorch	6.52	21.43	148	0.3043
TensorRT (float32)	12.17	20.29	77	0.5999
TensorRT (int8)	18.01	17.00	48	1.0597
MNN	2.14	16.09	461	0.1330
MNN (int8)	2.13	16.06	463	0.1326

Table 7. RT-DETR inference efficiency on Jetson Orin NX.

Model	Runtime	E_{frame} (J/frame)	L_C (ms/GFLOP)	E_C (mJ/GFLOP)
YOLOv8l	Pytorch	2.335	0.707	14.12
YOLOv8l	TensorRT (float32)	1.949	0.550	11.79
YOLOv8l	TensorRT (int8)	1.004	0.333	6.07
RT-DETR-l	Pytorch	3.287	1.370	30.43
RT-DETR-l	TensorRT (float32)	1.667	0.713	15.44
RT-DETR-l	TensorRT (int8)	0.944	0.444	8.74

Table 8. Derived explanatory metrics on Jetson Orin NXDerived explanatory metrics on Jetson Orin NX.

Model	$R_{compile}$	R_{INT8}	S_{conv}	S_q
YOLOv8l	1.29×	1.65×	0.57%	3.42%
RT-DETR-l	1.92×	1.61×	3.24%	2.96%

Table 9. Decomposition of optimization and accuracy-retention effectsDecomposition of optimization and accuracy-retention effects.

Conclusion

This paper evaluated the large object detectors YOLOv8l and RT-DETR-l across heterogeneous edge deployments, considering detection accuracy, model-execution latency, end-to-end throughput, and energy efficiency under realistic video-processing conditions. The results show that deployment suitability cannot be inferred from nominal model size alone, but must instead be interpreted jointly across model architecture, runtime, hardware target, and post-training deployment pathway.

On Raspberry Pi 5, CPU-based execution of large detectors remains impractical for real-time video processing because latency stays in the multi-second range. NPU-assisted paths improve FPS/W, with HailoRT as the strongest for YOLOv8l (Fig. 10). However, these gains are not uniformly transferable across accelerators: the Edge TPU path requires reduced input resolution and INT8-only execution, which materially lowers detection accuracy, whereas HailoRT preserves a more favorable balance between efficiency and precision under the tested conditions.

On Jetson Orin NX, TensorRT is the most effective deployment runtime for both evaluated architectures. At the same time, the measured ranking is runtime-dependent rather than architecture-intrinsic alone: YOLOv8l is faster under PyTorch, whereas under TensorRT the ranking reverses and RT-DETR-l achieves the highest throughput among the evaluated Jetson configurations. The normalized analysis further indicates that YOLOv8l

maintains lower latency and lower energy per nominal unit of workload together with stronger cross-runtime accuracy stability, while RT-DETR-l benefits more strongly from TensorRT optimization and reaches the lowest measured energy per processed frame only after full TensorRT INT8 deployment.

Overall, the results show that deployment efficiency is determined jointly by nominal workload, runtime realization, and accuracy retention after conversion and quantization. No configuration reached ≥ 25 FPS real-time. For edge deployments with hard real-time constraints, further hardware-specific optimization, reduced pipeline overhead, and/or smaller detector variants therefore remain necessary.

Data availability

All datasets used in this research, including the COCO2017 validation set and the MOT16-13 benchmark video, are publicly available through their respective official repositories.

Received: 23 December 2025; Accepted: 25 March 2026
Published online: 27 March 2026

References

1. Singandhupe, A. & La, H. M. A Review of SLAM Techniques and Security in Autonomous Driving, *2019 Third IEEE International Conference on Robotic Computing (IRC)*, Naples, Italy, pp. 602–607 (2019). https://doi.org/10.1109/IRC.2019.00122
2. Wu, X., Li, W., Hong, D., Tao, R. & Du, Q. Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A survey. *IEEE Geoscience and Remote Sensing Magazine* **10**(1), 91–124. https://doi.org/10.1109/MGRS.2021.3115137 (2022).
3. Ham, A., Similien, D., Baek, S. & York, G. Unmanned Aerial Vehicles (UAVs): Persistent Surveillance for a Military Scenario, *International Conference on Unmanned Aircraft Systems (ICUAS)*, Dubrovnik, Croatia, 2022, pp. 1411–1417 (2022). https://doi.org/10.1109/ICUAS54217.2022.9836099
4. Chen, J. & Ran, X. Deep Learning With Edge Computing: A Review. *Proceedings of the IEEE* **107**(8), 1655–1674. https://doi.org/10.1109/JPROC.2019.2921977 (2019).
5. Askerbekov, D. et al. Embracing drones and the internet of drones systems in manufacturing – An exploration of obstacles. *Technol. Soc.* **78**, 102648 (2024).
6. Lazarevich, I. et al. YOLOBench: Benchmarking Efficient Object Detectors on Embedded Systems, *IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, Paris, France, 2023, pp. 1161–1170 (2023). https://doi.org/10.1109/ICCVW60793.2023.00126
7. Varghese, R. S. M., YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness, *International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*, Chennai, India, 2024, pp. 1–6, Chennai, India (2024). https://doi.org/10.1109/ADICS58448.2024.10533619
8. Vaswani, A. et al. Attention is all you need, in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Long Beach, California, USA, pp. 6000–6010. (2017).
9. Zhao, Y. *IEEE/CVF Conference on Computer Vision and Pattern* Recognition *(CVPR)*, Seattle, WA, USA, 2024, pp. 16965–16974, (2024). https://doi.org/10.1109/CVPR52733.2024.01605
10. Alqahtani, D. K., Cheema, M. A. & Toosi, A. N. Benchmarking deep learning models for object detection on edge computing devices, in *Lecture notes in computer science*, pp. 142–150. (2024). https://doi.org/10.1007/978-981-96-0805-8_11
11. Li, Y., Miao, N., Ma, L., Shuang, F. & Huang, X. Transformer for object detection: Review and benchmark. *Engineering Applications of Artificial Intelligence* **126**, 107021–107021. https://doi.org/10.1016/j.engappai.2023.107021 (2023).
12. Thin Jun, E. L., Tham, M. L. & Kwan, B. H. A Comparative Analysis of RT-DETR and YOLOv8 for Urban Zone Aerial Object Detection, *IEEE International Conference on Automatic Control and Intelligent* Systems *(I2CACIS)*, Shah Alam, Malaysia, 2024, pp. 340–345, Malaysia, 2024, pp. 340–345, (2024). https://doi.org/10.1109/I2CACIS61270.2024.10649836
13. Miglionico, G. C., Di Rienzo, F., Ducange, P., Marcelloni, F. & Vallati, C. A Comparative Analysis of Models for Real-Time Personal Protective Equipment Detection on Edge Devices, International Joint Conference on Neural Networks (IJCNN), pp. 1–8, Jun. 2025, (2025). https://doi.org/10.1109/ijcnn64981.2025.11227764
14. Rey, L. et al. A performance analysis of you only look once models for deployment on constrained computational edge devices in drone applications. *Electronics* **14**(3), 638. https://doi.org/10.3390/electronics14030638 (2025).
15. Mela, J. L. & Sánchez, C. G. Yolo-based power-efficient object detection on edge devices for USVs. *Journal of Real-Time Image Processing* https://doi.org/10.1007/s11554-025-01682-2 (2025).
16. Rosende, S. B., Ghisler, S., Fernández-Andrés, J. & Sánchez-Soriano, J. Implementation of an Edge-Computing vision system on Reduced-Board computers embedded in UAVs for intelligent traffic management. *Drones* **7**(11), 682. https://doi.org/10.3390/drones7110682 (2023).
17. Fan, Y., Zhang, L. & Li, P. A lightweight model of underwater object detection based on YOLOV8N for an edge computing platform. *Journal of Marine Science and Engineering* **12**(5), 697. https://doi.org/10.3390/jmse12050697 (2024).
18. Sonnara, F., Chihaoui, H. & Filali, F. Efficient real-time license plate recognition using deep learning on edge devices. *Journal of Real-Time Image Processing* https://doi.org/10.1007/s11554-025-01738-3 (2025).
19. Wijaya, C., Andriyadi, A., Chen, S. Y., Wang, I. J. & Yang, C. T. Comparison of YOLO Power Usage on Edge-AI Device with Smartmeter, in *Lecture notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, pp. 3–10. (2025). https://doi.org/10.1007/978-3-031-93825-2_1
20. Bhushan, C. M. et al. Deploying TinyML for energy-efficient object detection and communication in low-power edge AI systems. *Scientific Reports* **15**(1), 44299. https://doi.org/10.1038/s41598-025-27818-9 (2025).
21. Ruiz-Barroso, P., Castro, F. M. & Guil, N. Real-time unsupervised video object detection on the edge. *Future Generation Computer Systems* **167**, 107737. https://doi.org/10.1016/j.future.2025.107737 (2025).
22. Orășan, I. L., Seiculescu, C. & Caleanu, C. D. Benchmarking TensorFlow Lite Quantization Algorithms for Deep Neural Networks, *IEEE 16th International Symposium on Applied Computational Intelligence and Informatics (SACI)*, Timisoara, Romania, 2022, pp. 000221–000226 (2022). https://doi.org/10.1109/SACI55618.2022.9919465
23. Seshadri, K., Akin, B., Laudon, J., Narayanaswami, R. & Yazdanbakhsh, A. An Evaluation of Edge TPU Accelerators for Convolutional Neural Networks, *IEEE International Symposium on Workload Characterization (IISWC)*, Austin, TX, USA, 2022, pp. 79–91 (2022). https://doi.org/10.1109/IISWC55918.2022.00017
24. Coelho, B. L., Bodmann, P. R., Cavagnero, N., Frost, C. & Rech, P. Vision Transformer Reliability Evaluation on the Coral Edge TPU. *IEEE Transactions on Nuclear Science* https://doi.org/10.1109/TNS.2024.3513774 (2025).
25. Patuck, N. Introducing the Raspberry Pi AI HAT+ with up to 26 TOPS, Raspberry Pi, Oct. 24 (2024). Available: https://www.raspberrypi.com/news/raspberry-pi-ai-hat/
26. Ultralytics, N. C. N. N. *Ultralytics YOLO Docs*, Dec. **29** (2024). https://docs.ultralytics.com/integrations/ncnn/
27. Chen, Z., Chen, J., Ding, G. & Huang, H. A lightweight CNN-based algorithm and implementation on embedded system for real-time face recognition. *Multimedia Systems* **29**(1), 129–138. https://doi.org/10.1007/s00530-022-00973-z (2022).

28. Zhou, Y. & Yang, K. Exploring TensorRT to Improve Real-Time Inference for Deep Learning, *IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*, Hainan, China, 2022, pp. 2011–2018 (2022). https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys57074.2022.00299
29. Jocher, G., Chaurasia, A. & Qiu, J. Ultralytics YOLOv8. 2023. [Online]. Available: https://github.com/ultralytics/ultralytics
30. Shehzadi, T., Hashmi, K. A., Liwicki, M., Stricker, D. & Afzal, M. Z. Object Detection with Transformers: A Review. *Sensors* **25**(19), 6025. https://doi.org/10.3390/s25196025 (2025).

Acknowledgements
This work is supported by the Ministry of Defence of the Slovak Republic through the inter-ministerial sub-program 06E0I-Research and development in support of state defence.

Author contributions
Michal Turčaník: Conceptualization; Data curation; Formal analysis; Funding acquisition; Investigation; Methodology; Project administration; Resources; Software. Supervision; Validation; Visualization. Writing - original draft; Writing - review & editing. Ivan Suchý: Conceptualization; Data curation; Formal analysis; Investigation; Software; Validation; Visualization; Writing - original draft; Writing - review & editing.

Funding
This work was funded by the Ministry of Defence of the Slovak Republic.

Declarations

Competing interests
The authors declare no conflicts of interest and they have no known financial interests or personal relationships that could influence the work reported in this paper.

Additional information
Correspondence and requests for materials should be addressed to M.T.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026